学案 04 | PyTorch 张量、自动求导与训练循环

1. PyTorch 在本阶段做什么

PyTorch 是用于张量计算和机器学习的 Python 工具。本阶段只把它当作"数值计算器 + 可微分计算图 + 参数更新工具"，不训练大模型。

先运行 `代码/01_tensor_warmup.py`。在项目根目录 `agent` 打开的 PowerShell 中，可以直接指定本项目解释器来检查：

```
& '.\.venv\Scripts\python.exe' -c "import torch; print(torch.__version__);
print(torch.cuda.is_available())"
```

本阶段示例默认在 CPU 上运行，即使电脑有 GPU 也无须改设备代码。若编辑器提示找不到 `torch`，先按 环境使用说明 选择 `agent/.venv/Scripts/python.exe`；切换工作目录不会自动切换解释器。

2. 最常见的对象

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
print(x.shape)       # torch.Size([3])
print(x.dtype)       # 浮点数据类型
```

- `torch.tensor( ... )`：由 Python 数值建立张量。
- `shape`：每一维的长度；`[3]` 表示一维有 3 个数。
- `dtype`：每个值的数据类型。神经网络常用浮点数；类别编号则常用整数类型。
- `+`、`*`：对张量做数值运算。形状兼容规则留到后面按需学习。
- `@`：矩阵乘法运算符。
- `device`：数据所在的设备，例如 CPU 或 CUDA GPU。参与同一运算的张量通常需要在兼容设备上。

先学会每次重要运算后查看 `.shape`、`.dtype`。形状错误是初学深度学习最常见的问题之一。

3. `nn.Module` 与前向传播

PyTorch 通常用 `torch.nn.Module` 表示模型。模型类在 `__init__` 中声明需要长期保存的层，在 `forward` 中说明输入如何变成输出。

```
import torch.nn as nn

class SmallModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(in_features=1, out_features=1)
```

```
    def forward(self, x):
        return self.linear(x)
```

`nn.Linear(1, 1)` 有可学习的权重和偏置，数学上对应 `y = wx + b`。调用 `model(x)` 会由 PyTorch 调用 `forward`，并管理模块结构；通常不要手动直接调用 `model.forward(x)`。

初始化时参数通常是某种初始值，不会天然知道答案。训练循环通过损失与梯度逐步修改它们。

4. 自动微分（autograd）

若张量需要求梯度，设置 `requires_grad=True`。PyTorch 会记录由它参与的运算，并在对最终标量调用 `.backward()` 后计算梯度。

```
w = torch.tensor(1.0, requires_grad=True)
x = torch.tensor(2.0)
loss = (w * x - 5.0) ** 2
loss.backward()
print(w.grad)
```

这里表达式可写成 `loss=(2w-5)²`。在 `w=1` 时损失为 9，梯度为 -12。负梯度表示沿增大 `w` 的小方向移动会让损失下降。这个梯度是求导的局部结果，不是模型提供的自然语言解释。

在训练循环中，参数梯度默认会累积，所以每一批数据开始计算新梯度前要清零。优化器的 `zero_grad()` 通常负责这一动作。

5. 训练循环的五步

每个 batch 的基本顺序是：

```
1. optimizer.zero_grad()  清除前一批留下的梯度
2. prediction = model(x)  前向传播，产生预测
3. loss = loss_fn(prediction, target)  计算目标与预测的差异
4. loss.backward()        反向传播，计算参数梯度
5. optimizer.step()       按梯度更新参数
```

常见对象：

- `loss_fn`：定义“本次任务中怎样衡量错误”。回归可用 `nn.MSELoss()`；分类/下一个 token 预测常用 `nn.CrossEntropyLoss()`，但用法和目标形状要核对。
- `optimizer`：使用梯度实际更新模型参数，例如 `torch.optim.SGD` 或 Adam。它不是损失函数。
- `model.parameters()`：提供优化器要更新的可学习参数。
- `loss.item()`：把只有一个值的张量转成普通 Python 数，常用于打印记录。

最常见顺序错误是漏掉 `zero_grad()`，导致梯度跨 batch 累积；或漏掉 `step()`，导致算出了梯度却没有更新参数。

6. 推理模式的小提醒

训练与评估模式会影响某些层，例如 dropout 或 batch normalization。

- `model.train()`：设置训练模式。
- `model.eval()`：设置评估模式。
- `torch.no_grad()`：在不需要梯度时停止构建梯度图，节省内存和计算。

本阶段线性模型不含 dropout 或 batch normalization，所以 `train()` / `eval()` 的数值影响不明显；仍要先认识接口，阶段 1 阅读 nanoGPT 时会再次遇到。

7. 练习要求

1. 逐段运行 `代码/01_tensor_warmup.py`，记录每个张量的 shape。
2. 自己手算 warm-up 中 `loss=(w*x-5)²` 在 `w=1,x=2` 时的损失和梯度，并与运行结果对照。
3. 打开 `练习/01_补全训练循环.py`，仅参考注释补全关键空位；先不要打开答案。
4. 运行练习，确认训练后预测接近 `y=2x+1`，并检查 loss 的总体趋势。
5. 若结果异常，按"数据形状 → 前向输出 → loss → 梯度是否存在 → optimizer 是否更新"顺序排查。

8. 常见误解

- `loss.backward()` 只计算梯度，不会自动更新参数。
- `optimizer.step()` 才根据已计算的梯度更新参数。
- 训练损失下降不证明测试表现一定好。
- GPU 只是执行计算的设备，不会替代数据、目标函数或训练逻辑。
- Tensor 是多维数值容器，不意味着其中每个数都有自然语言语义。
- `eval()` 和 `no_grad()` 不是同一个功能：前者切换模块行为，后者控制是否记录梯度。